

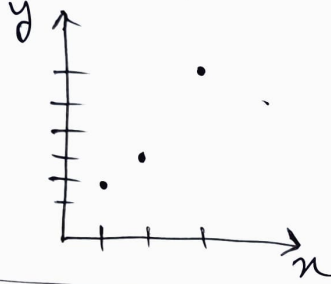
Linear Regression (HFN)

with Gradient Descent Optimization.

example, we have a 2 feature dataset where x is the independent variable and y is the dependant variable. let's say there are 3 data points ($m=3$), (x,y) are:

x	y
1	2
2	3
3	6

if we plot:



for linear regression, we are trying to find the best fit line. The final solution can look this, a linear plane ~~dividing~~ that demonstrates a linear correlation between two (or more) features.

Imagine the linear plane equation hypothesis is:

$$h_0(x) = \theta_0 + \theta_1 x \text{ [in the example]}, \text{ where } [\theta_1, \theta_0 \text{ are weights}]$$

we can see it as, $y = mx + c$

$$\Rightarrow f(x) = mx + c$$

here, the $\hat{y}$ value is basically the predicted y value from the model equation, not the actual value, so instead of y , we can write it as:

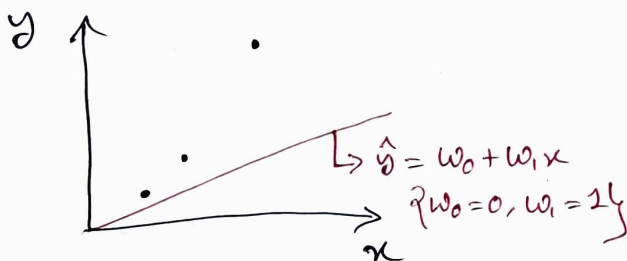
$\hat{y} = mx + c$; now x is the observed independent value/feature, so m & c is the value we need to update/change/approximate to get the best ~~fit~~ fit line. these parameters are known as weights. so the eqn is alternatively written as,

$h_w(x) = w_0 + w_1 x$

$\hat{y} = w_0 + w_1 x$

weights are $\{w_0, w_1\}$.

We have to adjust the weights to get the perfect value, an example:

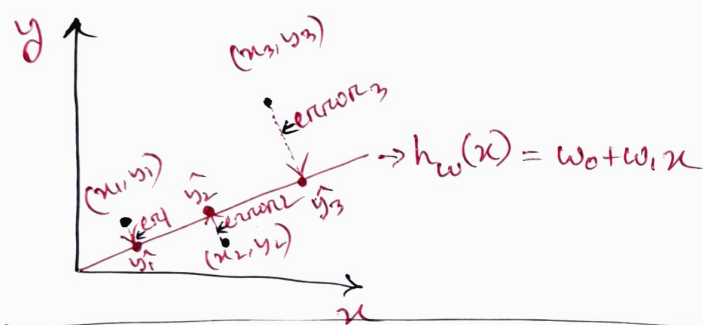


keep in mind, we are strictly talking about 2 feature data for 2D plot to better understand. It can be a problem with multiple features. i.e. features: $[x_1, x_2, x_3, \dots, x_n]$, target value y , then: $\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n$

Now, for the starting of your model approximation, you'll need a initial weight values. This can be done by giving default values [often given in the Problem Statement] or use Least Square method [given in the slide].

For our given Problem, Assume initially $w_0 = 0$, $w_1 = 1$; [as drawn in the last page]

Small Detour how we adjust the weight to approximate towards better result. Imagine, you are only able to change one weight at first (for simplicity), The error value calculation looks like this:



for the function, $y = w_0 + w_1 x$; the 3 data point of $x \Rightarrow (x_1, x_2, x_3)$, the predicted y values are $\{\hat{y}_1, \hat{y}_2, \hat{y}_3\}$. original value target is (y_1, y_2, y_3)

So, error/residual for each point $\Rightarrow (y_i - \hat{y}_i) \Rightarrow \boxed{e_i}$

To avoid positive/negative, we use Squared error $\Rightarrow (y_i - \hat{y}_i)^2 \Rightarrow \boxed{S.E_i}$

For all points $\Rightarrow$ Sum of S.E. $\Rightarrow \sum_{i=1}^m (y_i - \hat{y}_i)^2 \Rightarrow \boxed{S.S.E.}$

Mean SSE $\Rightarrow \boxed{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2} \Rightarrow \text{MSSE}$ $\left\{ \begin{array}{l} m \text{ is the total data} \\ \text{points} \end{array} \right.$

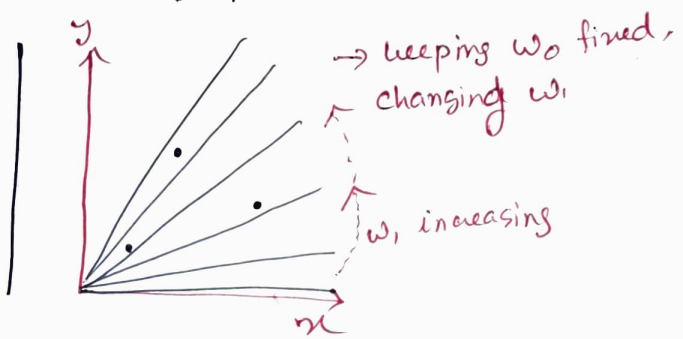
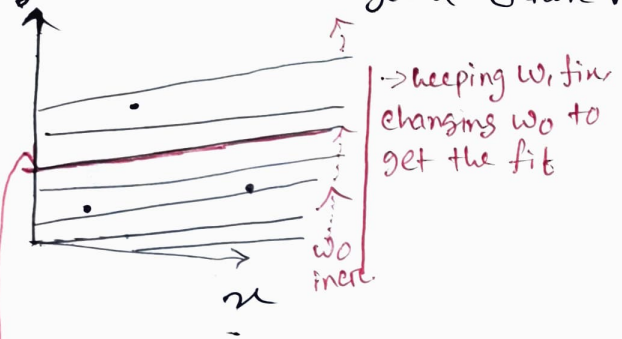
For mathematical conveniences [in the future] we write it as,

error function $\Rightarrow J(w_0, w_1) = \boxed{\frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2}$
/cost Function

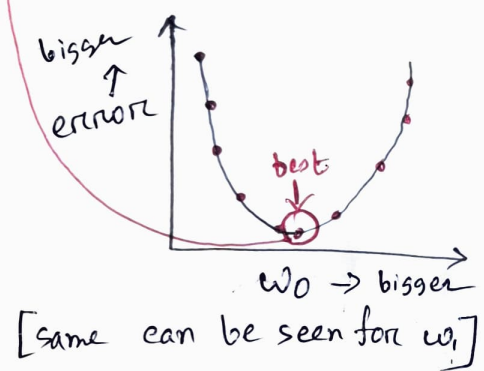
$$\Rightarrow \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

$$\Rightarrow \frac{1}{2m} \sum_{i=1}^m (\boxed{w_0 + w_1 x_i} - y_i)^2$$

now, knowing the error, if we isolate one weight and change the other, we will get a error vs weight graph similar to this.

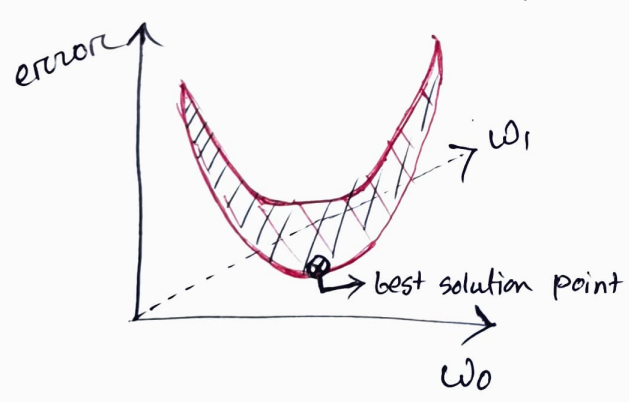


now, if we plot an ~~error~~ weight vs error curve:



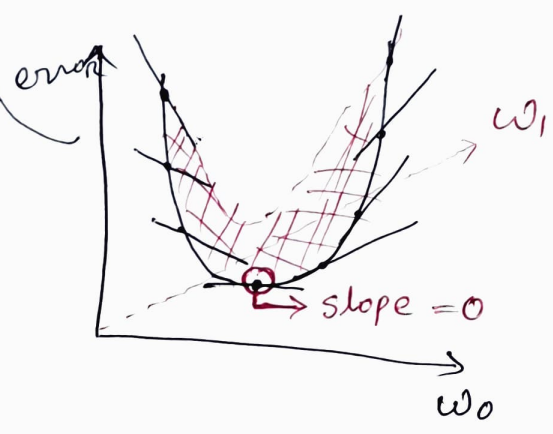
here, the marked point, is the value of w_0 where the error rate is lowest, we have to find that point.

But we have to visualize for both weights (w_0, w_1) for our case, so, the plotting is a 3D plain, looks like this:



In order to get there, instead of mindlessly searching, we can go according to the slope/derivative of this complex shape to get to the lowest point where the slope is closest to zero. The graph below shows this.

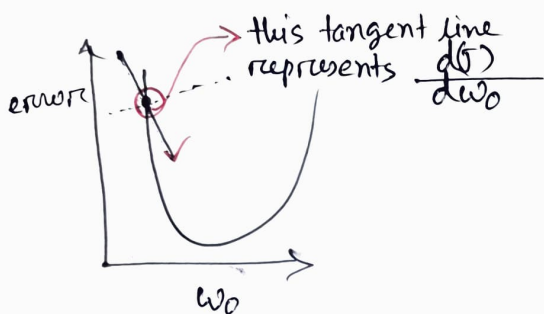
For that, we try to find the gradient (derivative) of the function and try to converge to the lowest point, it's called Gradient Descent.



Gradient Descent

(4)

we are updating the weights w_0 and w_1 in the model function by calculating $\frac{d(\text{Error})}{dw}$; we are representing cost function as $J(w_0, w_1)$ and the weights are $\{w_0, w_1\}$ so we have to partially derivate it to get the gradient for each weight. for example,



so, for $w_0 \Rightarrow$

$$\begin{aligned} \frac{dJ(w_0, w_1)}{dw_0} &= \frac{d}{dw_0} \left\{ \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 \right\} \\ &= \frac{d}{dw_0} \left\{ \frac{1}{2m} \sum_{i=1}^m (w_0 + w_1 x_i - y_i)^2 \right\} \\ &= \cancel{\frac{d}{dw_0}} \frac{1}{2m} \sum_{i=1}^m (w_0 + w_1 x_i - y_i) \times \frac{d}{dw_0} (\underbrace{w_0}_{1} + \underbrace{w_1}_{0} x_i - \underbrace{y_i}_{0}) \end{aligned}$$

$$= \frac{1}{m} \sum_{i=1}^m (w_0 + w_1 x_i - y_i) \times 1$$

$$= \frac{1}{m} \sum_{i=1}^m (w_0 + w_1 x_i - y_i)$$

$$= \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)$$

here α is called learning rate that determines how big/small change for next step so that it doesn't get past local minima

$$\boxed{\frac{dJ}{dw_0} = \frac{1}{m} \sum_{i=1}^m e_i}$$

; similarly, calculating $\frac{dJ}{dw_1}$; same way above,

so, we update such as,

$$\boxed{\frac{dJ}{dw_1} = \frac{1}{m} \sum_{i=1}^m e_i \cdot x_i}$$

$$\boxed{w_{\text{next}} = w_{\text{now}} - \alpha * \frac{dJ}{dw_0} \quad \left| \quad w_{\text{next}} = w_{\text{now}} - \alpha * \frac{dJ}{dw_1} \right.}$$

(5)

For the given math, data point

$$\begin{matrix} (1,2) & (2,3) & (3,6) \\ x_1, y_1 & x_2, y_2 & x_3, y_3 \end{matrix}; \text{ model hypothesis linear function } \Rightarrow h_w(x) = w_0 + w_1 x \quad \{ \underline{w_0 = 0}; \underline{w_1 = 1} \}$$

$$\text{cost Function: } J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m (h_w(x_i) - y_i)^2$$

$$\text{learning rate, } \alpha = 0.1 \quad = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2; m=3$$

① compute all predictions $\hat{y}$ given datapoint,

$$\rightarrow \hat{y}_1 = \frac{w_0}{0} + \frac{w_1}{1} * \frac{1}{x_1} = 1; \text{ target value } y_1 = 2; \text{ error} = e_1 = \hat{y}_1 - y_1 = -1$$

$$\rightarrow \hat{y}_2 = 0 + 1(2) = 2; \quad " \quad " \quad y_2 = 3; e_2 = -1$$

$$\rightarrow \hat{y}_3 = 0 + 1(3) = 3; \quad " \quad " \quad y_3 = 6; e_3 = -3$$

$$\hat{y} = \{2, 3, 6\}$$

② squared error $\Rightarrow e_i^2 \Rightarrow \{1, 1, 9\}$

③ cost function $\Rightarrow \frac{1}{2m} \sum_{i=1}^m e_i^2 = \frac{1}{6} (1+1+9) = 1.833$

④ gradient descent;

$$\boxed{\frac{dJ}{dw_0}} = \frac{1}{m} \sum_{i=1}^m e_i = \frac{1}{m} (-1-1-3) = \frac{-5}{3}$$

$$\boxed{\frac{dJ}{dw_1}} = \frac{1}{m} \sum_{i=1}^m e_i * x_i = \frac{1}{3} \{ (-1 \times 1) + (-1 \times 2) + (-3 \times 3) \} = \frac{-12}{3}$$

$$\text{so, } w_{0 \text{ next}} = w_{0 \text{ now}} - \alpha * \left(\frac{-5}{3} \right)$$

$$= 0 - 0.1 \left(\frac{-5}{3} \right)$$

$$= \underline{0.1667}$$

$$w_{1 \text{ next}} = w_{1 \text{ now}} - \alpha \left(\frac{-12}{3} \right)$$

$$= 1 - 0.1 \left(\frac{-12}{3} \right)$$

$$= \underline{1.4}$$

now, repeat the process with the updated $\{w_0, w_1\}$.

(6)

[There's a concept of decay, such that learning rate $\alpha = \text{decay factor} * \alpha$ where decay factor is a function of time where after each iteration, the learning rate gets a bit smaller. [initial step $t=0$; after each w_0, w_1 update; update the learning rate accordingly before doing the next update.]

* learn stochastic GD, mini batch GD, learning rate speed affect in GD (*)